AI STOCK MARKET PREDICTION

NSE Equity Price Forecasting, Trading Signals, News Sentiment & Walk-Forward Backtesting

Item	Detail
Problem	Predict tomorrow's closing price of any stock and derive a BUY / HOLD / SELL signal
Models	XGBoost (x4) + LSTM + GRU + Inverse-RMSE weighted Ensemble
Data	Yahoo Finance via yfinance (prices) + RSS news (sentiment)
Sentiment	FinBERT transformer (offline-cached) with VADER fallback
Validation	Chronological splits + walk-forward backtest (no look-ahead)
Interface	Single Streamlit web app (dark theme) with 5 pages
Stack	Python 3.11, pandas, numpy, scikit-learn, XGBoost, TensorFlow/Keras, Streamlit, Plotly

Academic disclaimer
This is a college major project. It is NOT investment advice. Stock markets are inherently unpredictable; a model that works historically is not guaranteed to work in the future. Never invest real money based only on this tool.

This document explains the complete working of the project for a reader with no prior knowledge: every page of the app, every model, every calculation, and exactly how the news sentiment engine works.

1. What This Project Does

You type the stock symbol of any company (for example **TCS.NS** for Tata Consultancy Services on the Indian NSE exchange). The app then:

- **downloads** that company's full daily price history from Yahoo Finance (up to 10 years),
- **builds features** -- numerical summaries of price behaviour (averages, momentum, volatility, etc.),
- **trains** several machine-learning models on the spot (about 50 seconds the first time for a company),
- **predicts** tomorrow's return and converts it into a **BUY / HOLD / SELL** signal,
- **reads the news** headlines about the company with a financial-language AI (FinBERT) and lets that sentiment softly adjust the signal,
- **proves itself** with a walk-forward backtest -- a simulation of what would have happened in the past if the signal had been traded, including trading costs, compared against simply buying and holding.

Nothing is pre-trained for a fixed list of companies. Training happens **on demand**, only for the ticker you ask for, and the result is cached in the `saved_models/` folder so revisits are instant.

Why predict a return and not a price directly?

A share price can be Rs.50 or Rs.50,000 -- raw price values are impossible to compare across companies and drift endlessly upward. Models here predict the **percentage return for the next day**:

```
return(t)  =  price(t+1) / price(t)  -  1        # e.g. +0.012 = +1.2% tomorrow
```

The predicted price is then reconstructed as `predicted_price = current_price x (1 + predicted_return)`. This one design decision is what kept the old project's error (MAPE) at a realistic ~1.1% instead of 8%+.

2. How It Works at a Glance

The whole system is a six-step pipeline. Each step is explained in detail later:

Step	What happens	Code module
1. Data	Download OHLCV prices for the ticker + NIFTY 50 + India VIX; fetch news headlines via RSS	data/data_loader.py, data/news_loader.py
2. Features	Turn raw prices into 16 numerical features + forward-return targets	features/feature_engineering.py
3. Models	Train XGBoost (x4), LSTM, GRU; combine them with inverse-RMSE weights	models/train.py, models/sequence.py, models/ensemble.py
4. Signal	XGBoost 1-day return vs +/-0.5% threshold -> BUY/HOLD/SELL, tilted by news sentiment	models/predict.py
5. Backtest	Walk-forward simulation: retrain every 21 trading days, pay 10 bps per trade, compare vs buy & hold	models/backtest.py
6. Log	Store each prediction in SQLite; score direction accuracy once actuals arrive	utils/prediction_log.py

```
ticker -> yfinance data -> features -> [XGBoost 1d/5d/21d + direction] [LSTM] [GRU]
                                       \____________ inverse-RMSE ensemble ________/
                                                    |
                  news RSS -> FinBERT score -------+--> BUY / HOLD / SELL signal
                                                    |
                                walk-forward backtest + prediction log
```

3. Basic Concepts (for First-Time Readers)

Term	Meaning
Ticker	A symbol Yahoo Finance uses to identify a company. Indian NSE stocks end in **.NS** (TCS.NS, INFY.NS, RELIANCE.NS). Indices use **^** (^NSEI = NIFTY 50, ^INDIAVIX = India VIX). US tickers work too (AAPL).
OHLCV	For every trading day: Open, High, Low, Close prices and Volume (shares traded). The chart page draws these as candlesticks.
Return	Percent change between two closes: `close(t)/close(t-1) - 1`. Returns, not prices, are what the models learn.
Train / Test	Models learn from older data (train) and are scored on newer data they have never seen (test). Our split is strictly chronological -- the last 15% of history is the test set. No shuffling, because time order matters.
Overfitting	When a model memorises past noise instead of learning patterns; it scores great on training data but fails on new data. Guarded against by chronological testing, walk-forward validation, and comparing against a naive baseline.
Look-ahead bias	The worst sin in finance modelling: accidentally letting the model see the future while training. Prevented everywhere: scalers fit on training data only, features use only past values, backtest retrain steps use only data available on that day.
Naive baseline	The deliberately dumb predictor 'tomorrow's price = today's price'. Every metric is reported next to this baseline; a model that cannot beat it has no real skill.

4. Step 1 -- Where the Data Comes From

Price data: yfinance

`yfinance` is a free, unofficial Python library that reads Yahoo Finance. One call returns a table of daily OHLCV for the requested ticker and history length (1y / 2y / 5y / 10y). The adjusted close (price corrected for splits and dividends) is used as **Price**.

Three series are downloaded: the **stock itself**, **NIFTY 50 (^NSEI)** as the market benchmark, and **India VIX (^INDIAVIX)** as a fear gauge. All downloads are cached as CSV files under `data/cache/` for 24 hours, so repeated page loads do not hammer Yahoo and the app still renders when a fetch hiccupes.

News data: RSS feeds (no API key needed)

Two public RSS feeds are read with plain HTTP + XML parsing (no paid API):

- Yahoo Finance RSS, per ticker: `feeds.finance.yahoo.com/...headline?s=TCS.NS...`
- Google News RSS, searched by the company's real name (resolved once from Yahoo), which finds local/business articles RSS might miss.

Headlines are de-duplicated and limited (typically 30-75 recent items). This is the raw material for the sentiment engine described in Section 8.

5. Step 2 -- Feature Engineering

Models do not see prices; they see **features** -- 16 numbers per day that summarise the recent state of the market. Think of them as the model's instruments and gauges.

Feature	How it is calculated	Plain-English meaning
Return	price(t)/price(t-1) - 1	Today's percentage move
MA10	10-day rolling average of price	Short-term trend line
MA20	20-day rolling average of price	Medium-term trend line
Volatility	10-day rolling std-dev of daily returns	How jumpy the stock is right now
RSI	14-day Relative Strength Index (0-100)	Overbought (>70) vs oversold (<30) momentum
MACD	EMA12 - EMA26 of price	Trend-following momentum line
MACD_signal	9-day EMA of the MACD line	Crossover trigger for MACD
BB_high / BB_low	20-day mean +/- 2 standard deviations	Bollinger volatility bands -- price position in range
Momentum	price(t) - price(t-10)	Rupee change over the last 10 days
Lag1 / Lag2 / Lag3	returns of 1, 2 and 3 days ago	Recent memory of daily moves
Excess_Return	stock return - NIFTY return same day	Did it beat the market today?
Mkt_Vol	10-day rolling std of NIFTY returns	Is the whole market nervous?
VIX	India VIX index level (forward-filled)	Market-wide fear gauge (high = fearful)

Targets -- what the models learn to predict

```
Target      = price(t+1) / price(t)  - 1      # next-day return (main)
Target_5d   = price(t+5) / price(t)  - 1      # one-week return
Target_21d  = price(t+21) / price(t) - 1      # one-month return
```

The last rows have no target yet (the future has not happened) -- they are dropped from training. The final row's feature vector is exactly what is fed to the trained model to forecast tomorrow.

> **Why add NIFTY and VIX features at all?**
> A stock's future depends on its own history AND the market mood. Excess_Return, Mkt_Vol and VIX let the model learn regime patterns -- for example 'high-VIX days tend to mean-revert' -- which is also how we compensate for not having years of historical news (see Section 8).

6. Step 3 -- The Machine-Learning Models

Four model families are trained for each ticker. All outputs are **returns**; prices are reconstructed afterwards.

6.1 XGBoost (gradient-boosted decision trees)

XGBoost builds many small decision trees in sequence; each new tree focuses on the mistakes of the trees before it. It is the champion for tabular (spreadsheet-like) data. We train four of them:

Model	Predicts	Used for
xgb_1d	next-day return	**the trading signal** + display target
xgb_5d	5-day return	'1 week expected %' table on the Overview page

Model	Predicts	Used for
xgb_21d	21-day return	'1 month expected %' table
xgb_direction	class UP (1) / DOWN (0)	direction classifier metric on Model Performance page

6.2 LSTM (long short-term memory) -- original sequence model

A recurrent neural network that reads a **window of the last 30 days of returns**, one day at a time, keeping a memory of what it has seen, and outputs a single number: the predicted return for day 31. Its MinMax scaler is fit on the training window only (no look-ahead).

6.3 GRU (gated recurrent unit) -- newly added

GRU is a newer, streamlined sibling of the LSTM. Where the LSTM uses three gates plus a separate cell-state pipeline, the GRU compresses this into just **two gates -- reset and update**:

- **Reset gate**: decides how much of the old memory to forget when reading the new day's return.
- **Update gate**: decides how much of the old memory to carry forward vs replace.

Fewer gates mean fewer parameters, so a GRU trains faster and needs less data than an LSTM while usually matching its accuracy on short windows like ours. It consumes the exact same 30-day scaled-return sequence as the LSTM and also emits one next-day return. In the trained TCS model it even ended up with the largest ensemble weight (0.37), a nice payoff for adding it.

6.4 Inverse-RMSE weighted ensemble -- newly added

Three price predictions are better than one -- if they are weighted by trust. After training, each model is scored on the held-out validation window; the weight of each model is the inverse of its error, normalised so the weights sum to 1:

```
weight(i)  = (1 / RMSE_i) / sum( 1 / RMSE_j  for all models j )

example:  xgb RMSE = 40, lstm RMSE = 30, gru RMSE = 29  (rupees)
  inv     = 0.0250 , 0.0333 , 0.0345
  weights = 0.27   , 0.36   , 0.37      (they sum to 1.0)

final_price = 0.27*xgb_price + 0.36*lstm_price + 0.37*gru_price
```

A lower-RMSE (more accurate) model automatically gets a bigger say. If any model fails to produce a number, it is dropped and the remaining weights are re-scaled.

> **Important design note -- what drives the BUY/SELL signal?**
> The signal comes ONLY from the XGBoost 1-day model -- the same model validated in the walk-forward backtest. Sequence models (LSTM/GRU) extrapolate momentum and can be over-optimistic, so their outputs are shown as reference target prices, not as the trade trigger. The ensemble price is likewise a display target.

7. Step 4 -- From Prediction to Trading Signal

The XGBoost 1-day expected return is compared against a +/-0.5% dead-zone threshold (0.005 as a fraction; this value was chosen from the backtest):

Predicted 1-day return	Model signal
greater than +0.5%	BUY
between -0.5% and +0.5%	HOLD

Predicted 1-day return	Model signal
less than -0.5%	SELL

Example: current price Rs.2,000 and the model predicts +0.8% (Rs.2,016) -- that is above the threshold, so the base signal is BUY. A prediction of +0.2% is inside the dead zone: HOLD.

Sentiment tilt

The base signal may then be softened by the news score (details in Section 8). Rules:

News score	Model signal	Final signal	Confidence
>= +0.3 (positive)	BUY	BUY	high (aligned)
<= -0.3 (negative)	BUY	**HOLD** (downgraded)	low (conflict)
>= +0.3 (positive)	SELL	**HOLD** (softened)	low (conflict)
<= -0.3 (negative)	SELL	SELL	high (aligned)
any weak score	anything	unchanged	medium

Note what sentiment is NOT allowed to do: it can veto a trade (push BUY to HOLD) but it can never create one -- news alone can never flip a SELL into a BUY.

8. Sentiment Analysis -- Complete Deep Dive

8.1 Why read the news at all?

Prices react to information: an earnings blowout, a regulator fine, a CEO resignation. The price history alone cannot see tomorrow's headline, but reading today's headlines tells us the mood around a company. The app measures that mood as a single number (score) in [-1, +1].

8.2 Stage 1 -- collect headlines (RSS)

As described in Section 4: Yahoo Finance RSS for the exact ticker + Google News RSS for the company name. Typical yield: 30-75 recent headlines with title, date and source. No API key, no cost.

8.3 Stage 2 -- score each headline with FinBERT

FinBERT is a BERT language model -- a neural network pre-trained on huge amounts of English text, then further fine-tuned by researchers (ProsusAI) on financial writing. Because it is a transformer, it understands context and word order, which matters a lot in finance: it knows that 'profits fell' and 'losses fell' are opposite sentiments even though both contain similar words.

For each headline it outputs three probabilities -- P(positive), P(neutral), P(negative) -- which are turned into one score:

```
headline_score = P(positive) - P(negative)        # lies in [-1, +1]

example  'TCS Q4 profit jumps 40%, beats estimates'
         -> P(pos)=0.94 P(neu)=0.05 P(neg)=0.01  -> score = +0.93

example  'TCS shares fall as IT spending worries grow'
         -> P(pos)=0.02 P(neu)=0.11 P(neg)=0.87  -> score = -0.85
```

The model (about 400 MB) is downloaded **once** from Hugging Face and cached on disk; after that it runs fully **offline** (the code forces offline mode), so scoring adds no network delay and cannot hang on connection issues. Results are additionally cached in memory for 15 minutes per ticker.

8.4 Fallback -- VADER

If FinBERT cannot load (fresh machine, no cache), the app falls back to **VADER**, a classical rule-based sentiment tool: it scores words from a hand-built financial lexicon and applies grammar rules (negation like 'not good', intensifiers like 'very', capitalisation, exclamation). No AI weights -- pure rules -- so it is instant and dependency-free, though less accurate than FinBERT.

8.5 Stage 3 -- aggregate to one score per company

```
company_score = mean( headline_score for every headline fetched )

label:  score >= +0.3 -> 'Positive'   |   <= -0.3 -> 'Negative'   |   else 'Neutral'
```

The mean of typically 40-75 headlines is a robust mood measure -- one wild headline cannot dominate it. This company score is what feeds the tilt rules in Section 7. The Sentiment page shows it as a gauge from -100 to +100, the label, the number of headlines analysed, the engine used (finbert/vader), and every headline with its date and source.

> **Honest limitation (stated in the UI too)**
> Free RSS feeds only expose RECENT news, not a 10-year archive, so historical sentiment cannot be a training feature -- the model cannot learn 'bad news days crash' from data it never saw. Instead: (a) the trained models learn market mood from the NIFTY/VIX regime features, and (b) live news softly tilts the live signal, exactly as described.

9. Step 5 -- Walk-Forward Backtest

9.1 Why not just test on the last 15%?

That tests one fixed model. In reality you would retrain periodically as new data arrives. A **walk-forward** backtest mimics that real workflow and, crucially, can never peek at the future.

9.2 The algorithm (as implemented)

1. Take the full feature frame; drop rows with no target yet.
2. First **50%** of the days = initial training window; the remaining days are traded in the simulation.
3. Walk forward one day at a time. Every **21 trading days** (and at the start), refit a fresh XGBoost on **all data strictly before today** -- an expanding window, never including today or later.
4. Each day: predict tomorrow's return. Predicted return > +0.5% -> **position = 1 (long)**, otherwise **position = 0 (flat, sit in cash)**. No shorting.
5. Apply reality: every time the position flips (0 to 1 or 1 to 0), pay **10 bps (0.1%)** in costs.
6. Daily strategy return = position x that day's actual return - turnover x cost.
7. Compound everything into an equity curve and compare with buying the stock on day one and holding (buy & hold), starting from Rs.10,000.

9.3 The metrics shown on the Backtest page

Metric	Definition
Total return %	Final value / initial - 1, in percent
CAGR %	Compound annual growth rate: (final/initial)^(1/years) - 1
Sharpe ratio	mean(daily return) / std(daily return) x sqrt(252) -- return per unit of risk; higher is better
Max drawdown %	Worst peak-to-trough fall of the equity curve, in percent; the pain you would have felt

Metric	Definition
Trades taken	How many times the position flipped (each pays 10 bps)
Win rate %	Share of days the strategy was in the market and the day's return was positive
Directional accuracy %	Share of days the model predicted the correct up/down direction

A strategy that halves the drawdown of buy & hold while giving up some upside is doing exactly its job: risk control. (Real TCS example in Section 13.)

10. Step 6 -- Prediction Logging & Accuracy Tracking

Every time the dashboard opens a ticker (once per browser session), one row is written to a SQLite database (`data/prediction_log.db`):

```
ticker | predicted_on (next trading day) | current_price | predicted_price (XGB)
       | signal | model_signal | sentiment_score
```

When you open the app again on a later day, the actual close for the forecasted day is known, so the log resolves the row: was the predicted direction right or wrong? The Signal Log page shows predictions logged, how many have been resolved, and the running **direction accuracy** -- the project's own live scorecard, which needs no manual work and grows forever.

11. The App, Page by Page

Run `streamlit run app.py` and open http://localhost:8501. The app has a permanent sidebar plus five tabs.

Sidebar (visible on every page)

- **Stock ticker** -- any Yahoo symbol (TCS.NS, INFY.NS, RELIANCE.NS, ^NSEI, AAPL ...).
- **Training history** -- 10y / 5y / 2y / 1y of data to train on.
- **Load & Predict button** -- fetches + trains + shows the dashboard automatically. First time for a ticker shows a live progress bar (data fetch -> XGBoost 1d/5d/21d -> direction -> LSTM -> GRU -> ensemble -> save), about 50 seconds. Later visits are instant (cached model).
- **Model status card** -- when it was trained, on how many rows, over which date range.
- **Retrain button** -- forces a fresh training run (e.g. next day, to fold in new data).
- A bad ticker or dead connection produces a clear error with a 'Try again' button -- never a crash.

Page 1 -- Overview

- **Hero banner**: ticker, full company name (from Yahoo), current price and today's change % (last two closes).
- **Four target cards**: XGBoost / LSTM / GRU targets, each `current_price x (1 + that model's predicted return)`, plus the **ensemble target** = the inverse-RMSE weighted blend, with its expected return %.
- **Trading signal badge** (big coloured BUY / HOLD / SELL pill): the XGBoost 1-day return vs the +/-0.5% threshold, after the sentiment tilt. Under it: the model's raw signal, the sentiment label + score, and the direction classifier's UP/DOWN call.
- **Expected returns table**: 1-day, 5-day and 21-day return forecasts from the three XGBoost horizon models.
- **Price chart**: one year of candlesticks (each candle = one day's open-high-low-close) with MA10 (teal) and MA20 (gold) trend lines, and a volume histogram underneath.

Page 2 -- Model Performance

- **Test-set evaluation table** (last 15% of history, chronological): for XGBoost, LSTM, GRU and the Ensemble - RMSE in rupees, the naive baseline's RMSE, MAPE %, directional accuracy %, **skill = 1 - model_RMSE / naive_RMSE** (positive means it beats 'tomorrow = today'), and R-squared.
- **Multi-horizon table**: the 5-day and 21-day XGBoost models' validation metrics.
- **Direction classifier card**: accuracy of the UP/DOWN classifier next to the majority-class base rate -- the number to beat.
- **Ensemble weights chart**: the inverse-RMSE weights actually saved with the model.
- **Actual vs Predicted chart**: the test window with the real price (solid) and every model's forecast (dotted, ensemble in red).
- **Feature importance**: XGBoost 'gain' -- which of the 16 features mattered most to the trees, as a horizontal bar chart.

Page 3 -- Backtest

- Header metrics: strategy total return vs buy & hold, strategy max drawdown, Sharpe.
- Full comparison table: return, CAGR, Sharpe, max drawdown, final value of Rs.10,000.
- Trades taken, win rate (days in market), directional accuracy.
- **Equity curve**: two lines from Rs.10,000 -- the strategy and buy & hold -- over the test window. Everything computed by the walk-forward engine of Section 9 (refit every 21 days, 10 bps costs, no look-ahead).

Page 4 -- Sentiment

- **Gauge**: company score x 100, from -100 (very negative) to +100 (very positive), with the text label (threshold +/-0.3).
- **Headlines analysed** count and the engine used (finbert or vader fallback).
- **Impact box**: exactly how the tilt was applied -- 'model says X -> sentiment-adjusted signal is Y (confidence Z)'.
- **Headline list** with publication date and source, so you can verify the scores yourself.

Page 5 -- Signal Log

- Three summary cards: predictions logged, resolved (actual price now known), running direction accuracy %.

- Table of the last 30 logged predictions with their outcome -- the project's self-scoring scorecard (Section 10).

12. How to Run the Project

```
# 1. one-time setup
python -m venv .venv
source .venv/Scripts/activate      # Windows Git Bash
pip install -r requirements.txt

# 2. start the app
streamlit run app.py               # open http://localhost:8501

# optional: train tickers from the command line (multiple supported)
python train.py TCS.NS INFY.NS --period 10y

# tests
python -m pytest                   # 42 offline tests
python -m pytest -m network        # + 2 live end-to-end tests
```

In VS Code: select the .venv interpreter (Ctrl+Shift+P -> Python: Select Interpreter), then Run & Debug -> 'Streamlit app'.

13. Example Results & Honest Limitations

Results (TCS.NS, 10 years of data, 5-year test window)

- Forecast error: MAPE about 1.1% (naive-baseline-comparable at 1-day horizon -- daily noise dominates; this matches the well-known Efficient Market Hypothesis and is stated honestly).
- Ensemble weights: xgb 0.27 / lstm 0.36 / gru 0.37.
- Walk-forward backtest: the long/flat strategy roughly **halved the maximum drawdown** versus buy & hold by sitting in cash during the worst stretches, at the cost of some upside -- demonstrated risk control rather than guaranteed alpha.
- INFY.NS and WIPRO.NS trained on demand as further proofs of the any-ticker flow.

Limitations

- One stock's daily history is a small, noisy dataset -- predictions are indicative, never certain.
- No historical news archive -> sentiment tilts only the live signal.
- 10 bps cost and long/flat positions are simplifications (no intraday fills, slippage, taxes).
- Retraining is manual (button) or per page-load of a new ticker; there is no scheduled retrain.

Future scope ideas

- Sector/macro features and pair data.
- Transformer-based price sequence models.
- Optuna hyperparameter tuning; confidence-interval predictions.
- Scheduled daily retraining and alerting; paper-trading loop.

14. Project File Map

Path	Role
app.py	The entire Streamlit UI (sidebar auto-train flow + 5 pages)
train.py	Command-line training entry point
data/data_loader.py	yfinance downloads + 24h CSV cache
data/news_loader.py	RSS headline fetching + company-name resolution
features/feature_engineering.py	16 features + multi-horizon targets
features/pipeline.py	Builds the full feature frame (stock + NIFTY + VIX)
features/sentiment.py	FinBERT scoring (offline) + VADER fallback + caching
features/splits.py	Chronological and walk-forward split generators
models/train.py	Trains every model, computes ensemble weights, saves the bundle
models/sequence.py	LSTM/GRU training (train-only scaler, 30-day windows)
models/ensemble.py	Inverse-RMSE weights + weighted price blend
models/predict.py	Next-day inference, signal thresholds, sentiment tilt rules
models/backtest.py	Walk-forward simulation engine + metrics
models/metrics.py	RMSE / MAPE / directional accuracy / skill vs naive
models/registry.py	Per-ticker model bundles under saved_models/<ticker>/
models/explain.py	XGBoost feature importances
utils/prediction_log.py	SQLite predictions + accuracy resolution
tests/	42 offline unit tests + 2 live UI end-to-end tests
.streamlit/config.toml	Dark theme + server settings

Congratulations
If you have read this far, you can explain every screen, every model and every number in the app -- including exactly how news headlines become a graded trading signal. Good luck with the project demo!